Ders 4 Genişletilmiş Notu: Akış Diyagramı Örnekleri, Derleme ve Bağlama, C Temelleri, Veri Tipleri ve printf/scanf

Ders 4 Genişletilmiş Notu: Akış Diyagramı Örnekleri, Derleme ve Bağlama, C Temelleri, Veri Tipleri ve printf/scanf

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Çerçevesi ve Kaynaklar
- 2. Akış Diyagramı Sembolleri
- 3. for Döngüsü Örneği ve Algoritma Analizi
- 4. Şartlı Yapı Örneği
- 5. while Döngüsü Örnekleri
- 6. do-while Döngüsü, İşlem Kutusu ve Bağlantı Noktası
- 7. Birden Fazla Şart
- 8. İç İçe Döngü Örneği
- 9. Programlama Dilleri
- 10. Yüksek Seviyeli Dillerin Özellikleri ve Taşınabilirlik
- 11. Derleyici Kodu Nasıl İşler?
- 12. C Dili: Güçlü ama Tehlikeli
- 13. Yazılım Geliştirme Süreci, Köprü Metaforu ve Modülerlik
- 14. Derleme ve Bağlama
- 15. Tümleşik Geliştirme Ortamı (IDE)
- 16. Bir C Programının Temel Yapısı
- 17. Noktalı Virgül Kuralı
- 18. Değişkenler ve RAM Modeli
- 19. Değişken Adlandırma Kuralları
- 20. Veri Tipleri
- 21. Tiplerin Boyutları ve sizeof
- 22. Bildirim, Tanım ve Sabitler
- 23. Literaller
- 24. Giriş ve Çıkış Fonksiyonları
- 25. Gizli Tip Dönüşümü
- 26. Operatörler ve Operandlar
- 27. Büyük Tipten Küçük Tipe Dönüşüm ve Açık Dönüşüm
- 28. Değişkenler Bellekte Bitişik Tutulsa Daha mı Hızlı Olur?

1. Dersin Çerçevesi ve Kaynaklar

Hoca derse giriş konularına devam edileceğini söyleyerek başladı. Kullanılan slaytı zamanında Amaç Hoca ile birlikte hazırladıklarını, üzerinde bazı değişiklikler de yaptığını belirtti. Bu derste akış şemaları ve C programlama dili üzerine bilgi edinilecek, ardından sonraki haftalarda dersin asıl içeriğine geçilecekti. Hoca bunun nedenini şöyle açıkladı: Ders yalnızca bilgisayar bilimlerine ya da algoritmaya giriş olsaydı doğrudan akış diyagramı üzerinden anlatabilirdi. Ama bu ders yapısal programlamaya giriştir, yani C programlamayla ilgili şeyler de

yapmak gerekir. Bir şeyi tamamlamadan öbür tarafa geçmek iyi olmadığı için, eksik kalmasın diye araya ek konular alıyordu.

Geçen dönem C'de nereye kadar gelindiğini sordu. Cihan Hoca'nın grubundan bir öğrenci C'de stringlere yeni giriş yaptıklarını, Amaç Hoca'nın grubundan bir öğrenci ise C'ye pek girmediklerini, daha çok akış diyagramı üzerinden ilerlediklerini söyledi. Fonksiyon ve pointer konuları görülmemişti.

Kaynak olarak hoca birkaç şey saydı. Slaytta gösterdiği, ücretsiz olduğunu söylediği ve bağlantısını daha önce paylaştığını belirttiği bir kitap vardı. Kaufman'ın *Problem Solving and Program Design in C* kitabını da kaliteli bir kitap olarak andı. Bu kitapta özellikle bilgisayar grafiği üzerine güzel örnekler olduğunu, matematiksel düşünmenin yanında biraz da geometrik düşünmeyi istediğini söyledi. Ayrıca bölümün yapısal programlama dersinde kullanılan bir kitabı ve Coursera ile MOOC (Massive Open Online Course) platformlarındaki, örneğin Harvard'ın kaynaklarını önerdi.

> **Ders sonrası not:** Altyazıda yazar "Henry Kaufman" olarak geçiyor. Bu kitabın yazarları büyük olasılıkla Jeri R. Hanly ve Elliot B. Koffman'dır.

Önceki derslerde bilgisayar sisteminin nasıl çalıştığı ve nasıl bir mantığa sahip olduğu görülmüştü. Artık bir algoritmayla ve program geliştirerek bir problemin nasıl çözüleceğine geçiliyordu.

2. Akış Diyagramı Sembolleri

Hoca algoritmalarda üç temel yapı olduğunu hatırlattı: sıralı yapı, şartlı yapı ve tekrarlı (döngülü) yapı. Bunların yanında başlangıç ve bitişi, giriş-çıkış işlemlerini, kontrol ifadelerini, döngü ifadelerini ve atama ile karşılaştırma gibi temel ifadeleri gösteren semboller vardır. Bu semboller farklı kaynaklarda farklı şekillerde gösterilebilir. Aşağıdakiler Yıldız Teknik Üniversitesi'ndeki bu bölümde kullanılanlardır.

2.1 Başla ve Bitir

Yapısal programlama mantığında başlama ve bitiş kelimeleri önemlidir, çünkü yapısalcılık kuralları sıkı sıkıya tanımlamayı gerektirir. Pascal'da bir program `begin` ile başlar, `end` ve nokta ile biter. Arada `begin ... end;` yazılırsa orada bir blok başlamış olur. Bunun C'deki karşılığı süslü parantezlerdir. Bu, sıkı bir program geliştirme anlayışının yansımasıdır ve daha sonra nesne tabanlı dillere de yansımıştır.

2.2 Veri Girişi ve Çıkışı

Veri girişi kırık bir dikdörtgenle gösterilir. İçine, bir değer okunacaksa bir değişken adı, iki değer okunacaksa iki değişken adı, dizi söz konusuysa `A[i]` gibi bir eleman yazılır. Bu sembol, girilen değerlerin sırasıyla bu değişkenlere atanacağını gösteren bir notasyondur.

Kullanıcıya mesaj vermek ise başka bir iştir. Hoca bunu bir banka makinesiyle anlattı: Kartınızı makineye takacaksınız ve orada "kartınızı şu şekilde takınız" diye bir mesaj vardır. Öyle bir mesaj olmasa, konuya uzak olan insanlar şaşırabilir, yanlış yapabilir. Programda da "bir sayı giriniz" gibi bir şey söylenecekse önce bir çıkış sembolüyle mesaj verilir, sonra giriş sembolüyle değer okunur.

> **Hocanın vurgusu:** Veri girişi sembolünün içine "bir sayı giriniz A, B" gibi bir mesaj yazılmaz. İçine yalnızca değişken adı yazılır. Mesaj, ondan önce gelen ayrı bir çıkış işlemidir.

2.3 İşlem Kutusu

Dikdörtgen, atama ve matematiksel işlemler içindir: `A = 3`, `B = A + 2` gibi. Bunlar daha çok sıralı yapıda kullanılır.

2.4 Şartlı Yapı

Programın bir kısmı çalışacak, bir kısmı çalışmayacaksa `if` yapısı kullanılır. Sembol, hocanın tarifiyle "kulaklarından uzatılmış bir altıgen"dir ve içine şart yazılır. Bir doğru (true), bir de yanlış (false) kolu vardır. Doğru durumunda yapılacak işlemler bir kola, yanlış durumunda yapılacak işlemler öbür kola yazılır. Kollar bir yerde birleşir ve akış aşağı doğru devam eder.

> **Hocanın vurgusu:** `if`'in kollarından biri yukarı, biri aşağı gitmez. Kollardan biri yukarı gidiyorsa o yapı bir döngüdür, `if` değildir. Böyle bir şey belki `while` ya da `do-while` ile kodlanabilir, ama `if` ile bir ilgisi yoktur.

Hoca bunu C tarafına da taşıdı: C'de `if` ve `goto` ile döngü kurulmayacaktır. `if`, mantıksal olarak yalnızca belli işlemleri yapıp belli işlemleri yapmamak için düşünülür.

2.5 Döngü Sembolleri

Tekrarlı yapılarda iki farklı durum vardır. Başlangıcı, bitişi ve adım sayısı belli olan döngü `for` sembolüyle gösterilir. Şekilde kapalı bir çevrim görülür ve bu çevrimin içine istenen ifadeler yazılır. Bir şarta bağlı döngü ise `while` sembolüyle gösterilir. Şart doğru oldukça döngü gövdesi çalışır. Şart baştan yanlışsa gövde hiç çalışmayabilir.

3. for Döngüsü Örneği ve Algoritma Analizi

Slayttaki ilk örnekte başlangıç değerleri verilmemişti. Hoca kendisi `A = 0` kabul etti. Bir öğrencinin "n'in değerini bilmiyoruz" demesi üzerine de `N = 5` aldı. Slaytta döngü değişkeni bir yerde büyük, bir yerde küçük yazılmıştı. Hoca bunun küçük `i` olarak okunmasını istedi.

Diyagramın adımları şöyledir:

1. `A = 0`, `N = 5` kabul edilir.
2. `i` değişkenine 1 atanır. Bu işlem yalnızca bir kez yapılır.
3. `i < N` ise gövdeye girilir, değilse döngüden çıkılır.
4. `A = A + i` yapılır.
5. `i`, `"*"`, `A` ve alt satır yazdırılır.
6. `i` bir artırılır ve 3. adıma dönülür.

Sınıfta önce "15" cevabı geldi. Hoca tek bir çıktı olmadığını, döngü olduğu için birden fazla çıktı bulunduğunu söyledi. Ardından `0, 1, 3, 6, 10, 15` önerildi. Bir öğrenci de en alttaki * işaretinin çarpma olmadığını, ekrana yalnızca görüntü olarak basıldığını fark etti. Hoca bunu doğruladı: Bu bir çarpma işlemi değil, bir string'tir. Sonra iki öğrenci sırayla çözüm denedi. Hoca çözümü değişkenleri izleyerek baştan yaptı:

`i`	`i < N`?	`A = A + i`	Ekran
1	evet	1	`1*1`
2	evet	3	`2*3`
3	evet	6	`3*6`
4	evet	10	`4*10`
5	hayır		

Ekran çıktısı dört satırdır. Döngüden çıkıldığında `i` değişkeninin değeri 5'tir. Bu aşamada `i` yazdırılırsa 5 görülür.

```
1*1
2*3
3*6
4*10
```

Hoca burada analiz kavramının altını çizdi. Bir algoritmayı anlamak istiyorsak onun analizini yapmamız gerekir. Devre teorisi dersinde bir devrenin ne yaptığını anlamak için hangi direnç üzerinden ne kadar akım geçtiğine, ne kadar gerilim olduğuna bakılır. Burada da hangi değişkenin hangi aşamada hangi değeri aldığı izlenir.

3.1 Bitiş Değeri Dahil mi?

Bir öğrenci çıktının `5*15` ile biteceğini söylemişti. Hoca, 5'in `N`'den küçük olmadığı, ona eşit olduğu için döngüye girilmediğini anlattı. Geçen dönem hocaların bitiş değerinin dahil edilebileceğini söylemiş olabileceğini, bu yüzden o çözümün yanlış da sayılamayacağını ekledi. Kendi görüşü ise bitişin dahil edilmemesi yönündedir. Dahil edilecekse notasyonda ayrıca eşittir işareti yazılmalıdır. Madem bir akış diyagramı notasyonu oluşturuldu, 5'in dahil olup olmadığı belirsizliği ortadan kaldırılmalıdır: Eşittir yazıldıysa 5 dahildir, yazılmadıysa dahil değildir.

3.2 Akış Diyagramı Notasyonu Üzerine Sorular

Soru: Geçen dönem akış diyagramında iki ifadeyi karşılaştırırken tek eşittir kullanıyorduk. Artık hep iki eşittir mi kullanacağız?

Cevap: Bir şartlı ifadenin içinde eşittir ya da eşit eşit yazmak anlam açısından pek fark etmez, ama eşittirin görevi çoğu yerde atamadır. Hoca C notasyonunu daha çok beğeniyor ve kendi sınavlarında `==` kullanılmasını bekliyor.

Soru: True ve false kollarının yerini değiştirebilir miyiz? Akış boyunca aynı tarafta tutmak gerekir mi?

Cevap: Değiştirebilirsiniz, bu önemli değildir. Kurguyu ona göre düzenlersiniz. Bir yerde true'yu sağa, başka bir yerde sola yazmak da serbesttir. Kod yazarken `if`'in true kolu önce gelmek zorundadır, ama akış diyagramında böyle bir zorunluluk yoktur. Hocanın alışkanlığı true'yu sağa, false'u sola yazmaktır.

Soru: Kolların üzerindeki okları çizmek zorunlu mu?

Cevap: Evet, neyin nereye gittiğini göstermek için zorunludur. Kollar arasına da böyle oklar eklenebilir.

4. Şartlı Yapı Örneği

Hoca aynı diyagramı iki ayrı başlangıç değeriyle çözdü. Diyagramın adımları şöyledir:

1. `A mod B == 0` şartına bakılır.
2. Şart doğruysa `K = A / B` hesaplanır, ekrana `A`, `"/"`, `B`, `"="`, `K` yazılır.
3. Şart yanlışsa ekrana `A`, mod işareti, `B` ve "sıfıra eşit değildir" anlamındaki metin yazılır.
4. Kollar birleşir, akış aşağı doğru devam eder.

Birinci durum, `A = 10, B = 4`: $10 \bmod 4 = 2$ olduğundan şart yanlıştır. Ekrana 10, mod işareti, 4 ve "sıfıra eşit değildir" metni yazılır. Kısa nottaki gösterimle `10%4!=0`.

İkinci durum, `A = 8, B = 4`: $8 \bmod 4 = 0$ olduğundan şart doğrudur. `K = 8 / 4 = 2` hesaplanır ve ekrana `8/4=2` yazılır.

İki durum birbirinden bağımsızdır: Birinci durumda yalnızca bir kol, ikinci durumda yalnızca öbür kol çalışır.

> **Hocanın vurgusu:** Şu hata çok yapılıyor, tekrar edeyim: `if`'in bir kolu yukarı, bir kolu aşağı gitmez. `if` ile döngü oluşturulmaz. `if`'te akış daima aşağı doğru gider.

False kolunda hiçbir şey yapılmayacaksa, o kol boş bırakılır ve ok aşağı doğru gider. Kod yazarken `else` tarafında bir şey yapılmayacaksa `else` yazmaya gerek yoktur. Ama akış diyagramında false kolu boş da olsa çizilmelidir.

Slaytta başka örnekler de vardı. Hoca bunları daha sonra slayttan çözmeyi öğrencilere bıraktı.

5. while Döngüsü Örnekleri

5.1 Sonsuz Döngü Örneği

Hoca başlangıç değerlerini `A = 8`, `B = 2` olarak verdi. Bir öğrenci `K` için de değer gerektiğini söyledi. `K` önce 1 denildi, sonra 0 olarak alındı.

1. `A != B` oldukça döngü çalışır.

2. `B = B * B` yapılır.
3. `K = K + 1` yapılır.
4. `K mod 2 == 0` ise `B`, değilse `A` yazdırılır.
5. Alt satıra geçilir, üç tire yazılır, yine alt satıra geçilir ve 1. adıma dönülür.

Bir öğrenci çözümü denerken ilk başta `B`'yi iki katına çıkardı, sonra kendisiyle çarpılması gerektiğini fark edip düzeltti. İkinci turun ardından da sonsuz döngüye girildiğinden şüphelendi. Hoca "Sonsuz döngü olabilir, ben olmayacak diye bir şey demedim" diyerek izlemeyi yaptı:

Tur	`A != B?`	`B`	`K`	`K mod 2`	Yazılan
1	$8 \neq 2$	4	1	1	`A`, yani 8
2	$8 \neq 4$	16	2	0	`B`, yani 16
3	$8 \neq 16$	256	3	1	`A`, yani 8
4	$8 \neq 256$	65536	4	0	`B`, yani 65536

```
8
---
16
---
8
---
65536
---
...
```

`B` değeri 2, 4, 16, 256, 65536 diye büyür ve hiçbir zaman 8'e eşit olmaz. Dolayısıyla şart her zaman doğrudur ve döngünün sonu yoktur.

> **Ek bilgi (derste anlatılmadı):** Akış diyagramında `B` sınırsız büyüyebilen bir sayı gibi düşünülür. Aynı diyagram C'de 32 bitlik `int` ile yazılsaydı, $65536 \cdot 65536 = 2^{32}$ değeri büyük olasılıkla `int` sınırını aşardı. C'de işaretli taşmanın davranışı da büyük olasılıkla standartça tanımlı değildir. `gcc -fwrapv` ile yapılan denemede `B` önce 0 oldu, sonra hep 0 kaldı. Yani döngü bu koşulda da bitmez, ama ekrandaki değerler diyagramdakinden farklılaşır.

5.2 Biten while Örneği

İkinci diyagramda aynı başlangıç değerleri kullanıldı: `K = 0`, `A = 8`, `B = 2`.

1. `A > B` oldukça döngü çalışır.
2. `B = B * B` ve `K = K + 1` yapılır.
3. `K mod 2 == 0` ise `B` yazdırılır.
4. Değilse `A = A + 1` yapılır ve `A` yazdırılır.
5. Alt satıra geçilir, üç tire yazılır, yine alt satıra geçilir ve 1. adıma dönülür.

Hoca önce birinci turun ardından döngüden çıkılacağını söyledi. Bir öğrenci `B`'nin 16 olmasının döngünün içinde gerçekleştiğini hatırlattı ve hoca izlemeyi düzeltti:

Tur	Şart	`B`	`K`	`K mod 2`	İşlem	Yazılan
1	$8 > 2$	4	1	1	`A = 9`	9
2	$9 > 4$	16	2	0		16
	$9 > 16$ yanlış				çıkış	

```
9
---
16
---
```

Son değerler `K = 2`, `A = 9`, `B = 16` olur. Bir öğrenci 16'nın altında da üç tire olması gerektiğini sordu. Hoca doğruladı: 16 yazıldıktan sonra tireler de yazılır.

6. do-while Döngüsü, İşlem Kutusu ve Bağlantı Noktası

6.1 Neden do-while?

`for` ve `while` sembollerinde şart doğru değilse döngü gövdesi hiç çalışmayabilir. Oysa bazen şart doğru da olsa yanlış da olsa gövdenin en az bir kez çalışmasını isteriz. Hocanın örneği bir sayfaya giriştir: Usis'e girecekseniz en az bir kez şifrenizi girmeniz gerekir. Doğru da girseniz yanlış da girseniz şifre girilir. Yanlış girerseniz sistem tekrar ister, bunun da bir limiti vardır.

Böyle bir durumda `for` ve `while` pek işe yaramaz ya da işe yaraması için ek değişkenler kullanıp şarta müdahale etmek, aşağıdaki kurguyu da değiştirmek gerekir. Güzel bir çözüm `do-while` döngüsüdür. Akış gelir, önce gövdedeki işlemler yapılır. Burada örneğin giriş işlemleri vardır. Sonra şarta bakılır. Şart doğruysa gövdedeki işlemler tekrar yapılır, yanlışsa aşağıdan devam edilir.

6.2 İşlem Kutusu ve Bağlantı Noktası

Slayttaki bir işlem kutusunda `B`'nin 2'ye göre modunun alınıp sonucun `A`'ya atanması gösteriliyordu. Bir blok içinde birden fazla işlem de yapılabilir: Önce `A`'yı bir artır, sonra `B`'yi bir azalt, sonra `C` ile çarp gibi. Bölme ve mod için slaytta belirli gösterimler vardı. Hoca bunların ikisinin de kullanılabileceğini, tam bölme gibi ayrıntılarla çok uğraşılmadığını söyledi.

Kağıtta yer kalmazsa, örneğin bir `if` bloğunun devamı başka yere taşınacaksa, numaralı bir çıkış konur: "Bu, 1 numaralı çıkışa gidecek." Program başka bir yerde aynı numaralı noktadan devam eder. Hoca burada ok işaretlerinin de bulunmasının iyi olacağını belirtti.

6.3 do-while Örneği: `A > B`

Başlangıç değerleri `i = 0`, `B = 1`, `A = 10`.

1. `A = A - i` yapılır.
2. `i = i + 3` yapılır.
3. `A > B` ise 1. adıma dönülür, değilse devam edilir.

Hoca izleme sırasında üçüncü turda `A`'dan yanlışlıkla 3 çıkarıp 4 buldu. Öğrenciler `i`'nin 6 olduğunu hatırlattı ve hoca düzeltti.

Tur	`A = A - i`	`i = i + 3`	`A > B`?
1	$10 - 0 = 10$	3	evet
2	$10 - 3 = 7$	6	evet
3	$7 - 6 = 1$	9	hayır, eşit

Son değerler `i = 9`, `B = 1`, `A = 1` olur.

6.4 do-while Örneği: `A < B`

Yine `i = 0`, `B = 1`, `A = 10` ile başlanır. Bu kez sıra farklıdır:

1. `i = i + 2` yapılır, yani `i` 2 olur.
2. `A = A - i` yapılır, yani `A` 8 olur.
3. `A < B` yani $8 < 1$ yanlıştır, döngüden çıkılır.

Son değerler `i = 2`, `B = 1`, `A = 8` olur. Şart yanlış olduğu halde gövde bir kez çalışmıştır. Aradaki fark budur: Şart doğru da olsa yanlış da olsa gövde en az bir kez çalışır.

7. Birden Fazla Şart

Hoca, döngülü işlemlerde birden fazla şart verilip verilemeyeceğini sordu. Cevap evettir: Şartlar `and` ya da `or` ile bağlanır. Bütün şartların sağlanması gerekiyorsa `and`, birinin sağlanması yeterliyse `or` kullanılır.

Üç şart olduğunda işler biraz değişir. Birinci şartın doğru ve yanlış olma durumu vardır, ikincinin ve üçüncünün de öyle. Bunların alt kümelerine bakılırsa $2^3 = 8$ farklı durum oluşur. Hangisinin doğru, hangisinin yanlış olması gerektiği `and` ve `or` ile, parantezler kullanılarak açıkça belirtilmelidir: "Birinci şart doğruysa veya ikinci şart yanlışsa ve üçüncü şart doğruysa" gibi. Hoca bunların Lojik Devreler ve Ayrık Matematik derslerinde görüleceğini söyledi.

Örnek olarak bir öğrencinin başarı durumunu aldı: Devamsızlığı 20'den az ve ortalaması 40'tan yüksekse başarılı olsun ya da işlem tekrarlansın. Şart şöyle yazılır:

$$\text{devamsızlık} < 20 \;\text{ ve }\; \text{ortalama} \geq 40$$

Aynı şart şu biçimde de yazılabilir:

$$\text{değil}\,(\text{devamsızlık} \geq 20 \;\text{ veya }\; \text{ortalama} < 40)$$

Hoca bu mantıksal işlemleri yazabilmenin kurgu açısından önemli olduğunu vurguladı.

> **Ders sonrası not:** Hoca eşdeğerliği ikinci kez okurken "devamsızlığı 20'den büyükse veya ortalaması 40'tan büyükse" dedi. Doğrusu, ilk söylediği ve yukarıda yazılan biçimdir: devamsızlık ≥ 20 veya ortalama < 40. İki yazımın aynı sonucu verdiği python ile tam sayı değerler üzerinde denendi.
>
> **Ek bilgi (derste anlatılmadı):** Bu dönüşümün adı büyük olasılıkla De Morgan kuralıdır: $\neg(p \vee q) = \neg p \wedge \neg q$. Yani bir "veya" ifadesinin değili alınınca her şartın değili alınır ve bağlaç "ve" olur. "Değil" alınırken `<` işaretinin `>=` olduğuna dikkat edilmelidir.

8. İç İçe Döngü Örneği

Son akış diyagramında dışta bir `do-while`, içte bir `while` vardı. Başlangıç değerleri `T = 0`, `Y = 5`, `A = 7`, `B = 3`, `i = 1`.

1. `A = A - i` yapılır.
2. `K = 2` yapılır.
3. `K <= Y` oldukça: `T = T * 2 + i`, `K = K + 3`, sonra `T` ve `";"` yazdırılır.
4. `i = i + 3` yapılır.
5. `A > B` ise 1. adıma dönülür, değilse devam edilir.

İzleme şöyledir:

Dış tur	`A`	`K` (başta)	`T` hesabı	`K` (sonra)	Yazılan
1	$7 - 1 = 6$	2	$0 \cdot 2 + 1 = 1$	5	`1;`
1		5	$1 \cdot 2 + 1 = 3$	8	`3;`
2	$6 - 4 = 2$	2	$3 \cdot 2 + 4 = 10$	5	`10;`
2		5	$10 \cdot 2 + 4 = 24$	8	`24;`

Birinci dış turun sonunda `K = 8` olduğu için iç döngüden çıkılır, `i` 4 olur ve $6 > 3$ doğru olduğu için başa dönülür. İkinci dış turun sonunda `i` 7 olur, $2 > 3$ yanlış olduğu için dış döngü de biter.

`1;3;10;24;`

Son değerler `i = 7`, `A = 2`, `B = 3`, `T = 24`, `Y = 5`, `K = 8` olur.

Soru: İkinci adımda `K <= Y` oldu, yani şart sağlandı. Döngüden çıkması gerekmiyor muydu? Ben şart sağlanınca döngüden çıkıldığını hatırlıyorum.

Cevap: Hayır. `while` "iken" demektir, yani şart doğru olduğu müddetçe döngü döner. Şart yanlış olunca çıkılır. `K` 5, `Y` 5 iken `5 <= 5` doğrudur, bu yüzden gövde bir kez daha çalışır. Hoca bir ayrıntı daha ekledi: Şart sağlanınca çıkılsaydı, `2 <= 5` iken döngüye hiç girilmezdi.

> **Hocanın vurgusu:** `while` döngüsünde şart ifadesi doğru olduğu müddetçe döngü döner. `do-while`'da da aynısı geçerlidir: Şart doğru olduğu müddetçe başa çıkılır, yanlışsa aşağıdan devam edilir.

9. Programlama Dilleri

Amaç problem çözmektir. Problemi çözerken bir algoritma kurulur. Bu algoritmanın artık sözde ya da kağıt üzerinde kalmayıp hayat bulması gerekir. Tasarlanan çözümü belli bir sistematik içinde gerçekleyen yazılımlar programlama dilleridir.

İlk başlarda, 1940'lar ve 1950'lerde, makine dili ve assembly dili ortaya çıktı. İkisi de makineye bağlıdır, yani işlemcinin içindeki komut setine bağımlıdır. Yazılan program ancak o işlemcinin komut setinde tanınıyorsa çalışır. Makine dilinde 0 ve 1'lerle kod yazmak zor olduğu için bunların birebir karşılığı olan assembly kullanılmaya çalışıldı. Ama bir yerden sonra bu da pek mümkün olmadı.

Bu yüzden günlük konuşma dilimize daha yakın olan yüksek seviyeli diller kullanılır: `A = B + C - 2` gibi. Günümüzde daha doğal diller de yavaş yavaş ortaya çıkıyor. Örneğin "öğrenciler tablosundan öğrencilerin isimlerini ve soy isimlerini getir" gibi bir ifade, günlük hayatta yazdığımız cümlelere çok benziyor.

Yüksek seviyeli diller belli bir mekanizmayla makine diline dönüştürülür. Bunu derleyici (compiler) ya da yorumlayıcı (interpreter) yapar. Derleyici programı tamamen derlemeye çalışır, yorumlayıcı ise satır satır işler. Hoca Sayısal Analiz dersinde gösterdiği Python ve MATLAB'ı yorumlayıcılı dillere örnek verdi: Bir satır çalışır, sonra öbür satır çalışır, yani satırlar tek tek yürütülebilir.

10. Yüksek Seviyeli Dillerin Özellikleri ve Taşınabilirlik

Yüksek seviyeli dillerin üç önemli özelliği sayıldı:

- **Taşınabilirlik:** Bir makinede yazılan kod başka bir makinede de çalışabilir, çünkü orada derlenebilir. Bir bilgisayar modeli için bir program, başka bir model için başka bir program yazmak gerekmez.
- **Okunabilirlik ve anlaşılabilirlik:** Dil günlük konuşma diline çok daha yakındır, içinde `for` ve `while` gibi ifadeler vardır.
- **Sürdürülebilirlik:** Programda değişiklik ya da hata ayıklama yapılacaksa, kod daha anlaşılır olduğu için bu işlemler daha kolaydır.

Bir de kısmi dezavantajı vardır. Makine diline dönüşürken belli bir hız ve bellek kaybı olur. Derleyiciler bunu optimize etmeye çalışır. Yine de her şeyi makine dilinde yazmak mümkün değildir. Gömülü bir sistemde çalışacak bir C programındaki sıralama algoritmasında bazı kısımlar assembly ile yazılırsa çalışma süresi (execution time) kısalabilir. Burada program yazmanın kolaylığı ile programın kısa sürede tamamlanması arasında bir ödünleşim (trade-off) vardır.

Soru: Bizim yazdığımız C kodlarında taşınabilirlik var mı?

Cevap: Var. Standart C'den söz ediyoruz. Windows'ta yazılan bir kod Linux'ta da derlenebilir, yani işletim sistemleri arasında taşınabilir. Öğrencinin yazıp gönderdiği kod hocanın bilgisayarında da derlenir, ek bir yapılandırma gerekmez. Hatta iki taraf da Windows kullanıyorsa çalıştırılabilir dosya bile taşınabilir. Ama bizim için önemli olan kaynak kodun taşınabilirliğidir.

Soru: Arayüz fonksiyonlarını kullanınca standartlık azalıyor. Örneğin terminalde renkli ifadeler kullanmak istediğimde kütüphaneler bir sürü işletim sistemine ayrı ayrı port çıkarmak zorunda kalıyor.

Cevap: Bunun için program daha parametrik yazılır. Program çalışırken önce işletim sistemini algılar. Windows ise Windows'a göre, Linux ise Linux'a göre terminal komutları çalıştırılır. İşletim sistemiyle yapılan etkileşimler elbette farklılık gösterir. Örneğin `system("cls")` yazıldığında çalışan şey aslında bir DOS komutudur. Hoca bunu komut satırı penceresini açıp `cls` yazarak gösterdi. Linux'ta bu komut bu şekilde yoksa hata alınır. Bunun için bir de şartlı derleme (conditional compilation) konusu vardır: Windows komutlarını içeren kısımlar ona göre derlenir ya da derlenmez.

11. Derleyici Kodu Nasıl İşler?

Bir A derleyicisi A bilgisayarı için, bir B derleyicisi B bilgisayarı için, bir başkası C bilgisayarı için makine kodu üretir. Yani aynı işi yapan kod her makine için kendi derleyicisiyle dönüştürülür.

Bu dönüşüm bir anda olmaz, aşama aşama gerçekleşir. Önce yazılan programdaki token'lar ayrıştırılır. Örneğin `A = B + 3` yazıldığında derleyici + operatörünün doğru kullanılıp kullanılmadığına bakar. Solunda `B`, sağında 3 vardır. `B`'nin 3 ile toplanabilir bir şey olması gerekir. Bu şekilde sözcüksel (lexical) ve anlamsal (semantic) analizler yapılır. Bunlar hesaplama kuramı ve derleyici tasarımı konularıdır. Kod gramer kurallarından geçtikten sonra makine koduna dönüştürülmeye biraz daha yakın hale gelir. `printf` ve `scanf` gibi önceden derlenmiş kodların ise yeniden derlenmesine gerek yoktur.

12. C Dili: Güçlü ama Tehlikeli

C tarih boyunca çok sık kullanılmış bir dildir. Hoca onu "neredeyse dillerin atası" diye niteledi. Bir sistem programlama dili olarak işletim sistemlerinde kullanılır, Unix ve B diliyle olan bağlantısı tarihçesinin parçasıdır. Derste ağırlıklı olarak ANSI C, yani C89/C90 standardı kullanılacaktır. Dev-C++ ya da Code::Blocks gibi ortamlarda, derste anlatılan biçimde kod yazmak yeterlidir. Visual Studio'nun standartları ise biraz farklı olduğu için önerilmez.

C'nin kullanım alanları çok geniştir:

- İşletim sistemlerinin yazılması.
- Mikrodenetleyiciler. Mikrodenetleyicilerin kendine ait komut setlerini öğrenmek çok zor olduğu için, PIC için olduğu gibi mikrodenetleyiciye özel C sürümleri oluşturulur. Bunlarda normal döngü ve diziler vardır, bunlara ek bazı özellikler bulunur.
- Telefonlar ve taşınabilir cihazlardaki gömülü işlemciler ile DSP cihazları.
- ADSL modemlerin ve switch'lerin içindeki çipler.

Özetle içinde bir işlemcisi ve belleği olan, küçük bir çipten koskoca bir bilgisayara kadar her şey C ile programlanabilir. Pascal da önceden kullanılıyordu, ama C'nin makine diline dönüşümü, notasyonu, hızı ve güvenliği daha kabul edilebilir bulundu. Java, C# ve C++ gibi diller de C'den geliştirildi.

Dilin yapısı sadedir. Anahtar kelime (keyword) sayısı azdır, dolayısıyla öğrenilmesi gereken kelime sayısı sınırlıdır. Hoca çok anahtar kelimenin dili belki kolay kullanılır yapsa da öğrenmeyi güçleştirdiğini söyledi. `int` ve `char` gibi ilkel (primitive) veri yapılarından `struct` ve `union` gibi bileşik yapılar tanımlanabilir. Dizilerle ya da doğrudan yollarla belleğe erişilebilir ve bu büyük bir güç kazandırır. Standart giriş-çıkış kütüphaneleri vardır, kod rahatça doğal (native) koda dönüştürülebilir. Makro işlemleri de mümkündür.

Hoca C için "güçlü ama tehlikeli" dedi. Tehlikeli olan yan şudur: Program derlenir, çalışır, ama diğer dillere göre yanlış sonuç alma olasılığı çok fazladır.

- **Dizi sınırı denetlenmez:** Beş elemanlı bir dizinin son elemanı 4 numaralı indekstedir, ama 5 numaralı elemana da erişilebilir. İşletim sistemi engel koyana kadar belleğe istenildiği kadar erişilebilir.
- **Aralık denetimi (range checking) yoktur:** Bir `int`'in sınırını programcı kontrol etmek zorundadır.
- **Çalışma zamanında tip denetimi (type checking) yapılmaz:** `int` girilmesi gereken yere string girilirse program iptal olabilir.
- **Tip programcıya bırakılmıştır:** `printf` ile bir şey bastırırken ya da `scanf` ile okurken verinin tipini kendiniz yazarsınız.
- **Gizli dönüşüm yapılır:** Kayan noktalı bir değer `int`'e atanırken hocanın aktardığına göre Pascal hata verir, C ise gizlice dönüştürür.

Bunların nedeni hızdır. Derleyici her işe karışmaz, temel kuralları koyar ve bu çerçevede program yazılmasını ister. Hoca bunu öğrenci mevzuatına benzetti: Mevzuat ne kadar sadeyse özgürlükler o kadar fazladır. Özgürlüklerin fazla olması ise öğrencinin yükünü artırır. C'de de her detayı programcı düşünmek zorundadır. Örneğin dizi dışına erişilmeye çalışıldığında Java doğrudan hata verir, C'de böyle bir durum yoktur.

13. Yazılım Geliştirme Süreci, Köprü Metaforu ve Modülerlik

Bir yazılım üretiminin aşamaları vardır:

1. **Gereksinimler:** Yazılımın gereksinimleri ve özellikleri belirlenir. Nasıl bir proje yapılacağı, nasıl bir yazılım ortaya konacağı bilinmelidir.
2. **Yapısal tasarım:** Örneğin hangi değişkenlerin kullanılacağı düşünülür.
3. **Proje planı:** İş bir proje haline getirilir ve planlanır.
4. **Detaylı tasarım:** Plana göre ayrıntılı tasarım yapılır. Hoca buna akış diyagramı diyebileceğimizi söyledi. İlk başta "dizi kullanırım" denir, detaylı tasarımda dizinin nasıl tanımlanacağı ve üzerinde hangi işlemlerin yapılacağı yazılır.
5. **Kodlama.**
6. **Hata ayıklama (debugging).**
7. **Test:** Hata yoksa test edilir. Testlerden başarıyla geçilirse ürün sunulur.
8. **Bakım:** Ürün sunulsa da sunulmasa da kod üzerinde sürekli güncelleme yapmak gerekir.

Hoca bunu bir köprü metaforuyla anlattı. Projeler büyüdükçe tasarım ve planlama daha ciddi çalışma gerektirir. Küçük bir inşaat ile Boğaziçi Köprüsü gibi bir yapı aynı değildir. Hataları önceden tespit edip uygun çözümler üretmek gerekir, çünkü ufak hatalar ileride felaket sonuçlara yol açabilir. Önceden hazırlanmış küçük parçaların (prefabricated parts) kullanılması da süreci iyileştirir. Köprü bir anda getirilip oraya konmaz, parça parça yapılır. Yazılım da modüler biçimde yapılacaktır.

Köprü ile yazılımın farkları da vardır. Köprü statik bir şeydir, ileride değişiklik yapılmaz. Bir apartmana "buna bir balkon çıkalım" denemez. Yazılımda ise bu mümkündür ve yazılım genellikle değişim için tasarlanır, çünkü gereksinimler sürekli yenilenir. Yazılımdaki her parçanın yeniden kullanılabilir olması gerekir. Bir de yazılım test edilebilir. Köprüde belki öyle bir test yapamazsınız, işi en başından sağlam yapmak zorundasınız.

13.1 Modülerlik Örneği

Yazılacak program C'nin gramer kurallarına uygun olacak ve tasarım olabildiğince modüler yapılacaktır. Dönem içinde önce yalnızca `main` ile yapılan işler, ilerledikçe ayrı fonksiyonlara bölünecektir. Modülerlik arttıkça parçaların uyumu artar ve aralarındaki bağlılık azalır.

Hocanın örneği şuydu: Bir diziyi sıralayıp ardından yazdıran tek bir fonksiyon yazdınız. Kullanıcı ise "dizimi sıralasın ama yazdırmasın" isteyebilir, çünkü belki bir milyon eleman sıralayacaktır. Ya da hiç sıralamadan yalnızca yazdırmak isteyebilir. Bu durumda kod kopyala-yapıştır yapılır ve yinelenen kod (duplicated code) problemi ortaya çıkar. Doğrusu, sıralamayı ayrı bir modülde, yazdırmayı ayrı bir modülde yazmaktır. Çağıran taraf ikisine de ihtiyaç duyarsa ikisini de çağırır.

> **Ders sonrası not:** Hoca "sadece yazdırmak isteyen için ayrıca sıralama kodu yazmanız gerekiyor" dedi. Bağlama göre büyük olasılıkla "yazdırma kodu" kastedilmiştir. Hoca hemen sonra da aynı örneği sıralama ve yazdırmayı ayırarak toparlıyor.

14. Derleme ve Bağlama

Aradan sonra hoca önce akış diyagramlarıyla ilgili gelen bir soruyu ele aldı.

Soru: Akış diyagramında hep `i = i + 1` gibi mi yazmalıyız, yoksa `i += 1` de olur mu?

Cevap: `i += 1` C'de olan bir şeydir, `i = i + 1` daha doğrudur. Hoca buna çok takılmaz, ama akış diyagramları veri yapıları ve algoritma analizi gibi başka derslerde de kullanılacağı için takılan hocalar olabilir. Ne olur ne olmaz, en doğrusu `i = i + 1` yazmaktır.

Sonra derlemenin nasıl olduğuna geçti. Bir program birden fazla kaynak koddan oluşabilir, modülerlik de bunu gerektirir. Her kaynak kod kendi içinde derlenir ve bir obje dosyası oluşur. Hoca bunun ayrıntısını sonraki derste bir örnek program üzerinden göstereceğini söyledi.

Örnek olarak şu kısa programı ele aldı:

```c
#include <stdio.h>

int main()
```

```
{
    int a;
    a = 3;
    a = a + 5;
    printf("%d\n", a);
    return 0;
}
```

`gcc -Wall` ile derlenip çalıştırıldığında ekrana 8 yazar. Hocaya göre `int a;`, `a = 3;` ve `a = a + 5;` satırlarından derleyici `mov` ve `add` gibi komutlar üretir, yani bu satırlar makine koduna hazır hale gelir. `printf` ise yeniden derlenmesi gerekmeyen, zaten derlenmiş bir şeydir. Onu tekrar derlemek uzun sürebilir, bu yüzden derlenmiş hali bulunduğu yerden getirilip ilgili kısma eklenir. İşte buna bağlama (linking) denir. Bu işi yapan programa bağlayıcı (linker) denir.

Süreç şöyle özetlenir:

1. Derleyici, kaynak kodu olabildiğince makine koduna çevirir ve obje dosyası oluşturur.
2. Bağlayıcı, derlenmiş başka kodları da bağlar. Bunlar `printf` olmak zorunda değildir, kendi yazdığınız ve derlediğiniz bir kütüphane de olabilir.
3. Ortaya çalıştırılabilir bir dosya çıkar. Bu `.exe` olabilir, başka dillerde Java'daki `jar` dosyası gibi biçimler görülür.
4. Dosya çalıştırıldığında belleğe bir örneği yüklenir ve proses haline gelir.

Hoca iki "süreç" kavramını karıştırmamayı istedi: Derlemenin aşamalarından oluşan süreç ile çalışan programı ifade eden proses aynı şey değildir.

> **Ders sonrası not:** Hoca obje dosyasının içinde `printf` ve belki `for` gibi C'ye tanıdık gelen şeyler göreceğimizi, tamamen assembly oluşmadığını söyledi. Obje dosyası büyük olasılıkla C kaynak metni içermez. `for` makine komutlarına dönüşmüştür, `printf` ise çözülmemiş bir sembol olarak kalır. Yukarıdaki program `gcc -c` ile derlenip `nm` ile incelendiğinde `printf` sembolü `U` (tanımsız) olarak görüldü, `objdump` çıktısında ise `movl` ve `addl` komutları vardı.

Soru: `printf` fonksiyonu daha önce var. Programı derleyince `printf`'in kendisi `.exe` dosyasının içine mi dahil oluyor, yoksa çalıştırırken sistem tarafından mı çağrılıyor?

Cevap: Birinci aşama derleme, ikinci aşama bağlamadır. Bağlama aşamasında derlenmiş kod, yani assembly ya da makine dilindeki karşılığı, çalıştırılabilir dosyanın içine yerleştirilir.

Soru: `printf` gibi çok büyük bir fonksiyonum olsa ve bunu iki programda kullanmak istesem, programın içine dahil etmeden dışarıdan çağırmanın yolu var mı?

Cevap: Hocaya göre `include` yapmadan dışarıdan çağıramazsınız, bu daha çok nesne yönelimli mekanizmalara yakın bir konudur. Yapılacak en fazla iş, `stdio.h` örneğinde olduğu gibi kendi dosyanızı `include` etmektir. Hoca ayrıca kaynak kodun boyutu ile derlenmiş kodun boyutunu karşılaştırdı: Kaynak metinden oluşur, bir karakter bir byte'tır, "bin satır yazsan bin byte'lık kod olur". Derlenmiş hali ise daha fazla yer tutar.

> **Ders sonrası not:** Hocanın bu iki cevabı statik bağlamayı anlatır. Büyük olasılıkla çoğu sistemde `printf` varsayılan olarak paylaşımlı bir kütüphanede (Windows'ta DLL, Linux'ta `.so`) durur ve program çalışırken ona bağlanılır. İkinci sorunun aradığı "dışarıdan çağırma" da büyük olasılıkla budur. Ayrıca `include` ile bağlama ayrı işlerdir: `stdio.h` büyük olasılıkla yalnızca bildirimleri içerir, derlenmiş kod bağlama aşamasında gelir. Hocanın "bin satır, bin byte" ifadesi de büyük olasılıkla bin karakter için geçerlidir.

Soru: Obje dosyası assembly dilinde mi oluyor?

Cevap: Tam assembly değildir. Hoca bunun sonraki derste gösterileceğini söyledi. Bilgisayarında komut satırından derleme için gereken ortam değişkenleri ayarlı olmadığından o an gösteremedi.

Soru: Java'nın çalışma zamanı kütüphanesiyle başka bir dilinki neden farklı? Obje dosyası dilden dile değişiyor mu?

Cevap: Evet, değişir. Derleme kuralları ve dillerin gramerleri farklıdır. Java'da derleyici kodu belli bir ölçüde `class` dosyasına dönüştürür. Bu dosya taşınabilirdir, başka bir yerde de kul-

lanılabilir. O makinedeki Java ortamı `class` dosyasını açar ve o bilgisayar için çalıştırılabilir hale getirir. C# ve .NET'te ya da başka dillerde mekanizma değişir, hepsinde aynı şey yoktur.

15. Tümleşik Geliştirme Ortamı (IDE)

Kaynak kodu yazmak, derlemek, hataları düzeltmek, varsa bağlamak, çalıştırmak, sonucu kontrol etmek ve adım adım hata ayıklamak gerekir. Bunları sürekli tek tek elle yapmak iyi olmaz. Hatta hata yapınca düzeltme önerileri de sunan bir yazılım olsa iyi olurdu. Hocanın dediği gibi: "Ben düşündüysem, yıllar önce bu çoktan düşünülmüştür." Bu işi yapan yazılıma IDE (Integrated Development Environment) denir.

Derste Dev-C++, Code::Blocks, Xcode ya da Eclipse kullanılabilir. Hoca Dev-C++ ve Code::Blocks'u, macOS'ta Xcode'u önerdi. Xcode'u kendisi kullanmamıştı. Kodların özellikle Code::Blocks ile derlenmesi iyi olur, çünkü standart farklılıkları görülebilir. Slayttaki tabloya göre Dev-C++ Windows ve Linux'ta, Code::Blocks Windows ve macOS'ta çalışıyordu.

Visual Studio'yu hoca C için pek önermedi: Çok ağır bir IDE'dir, güzel ve faydalıdır, kendisi C# ve web programlama anlatırken kullanır, ama bu ders için gerekli değildir. Visual Studio Code ise daha çok bir editördür, arkasında bir derleyicinin tanıtılması gerekir. Hoca burada IDE'lerden söz ettiğini, editör olarak Notepad gibi uygulamaların da kullanılabileceğini söyledi. Kendi örneklerini Dev-C++ ile yapacaktı.

> **Ders sonrası not:** Slayttaki platform bilgisi büyük olasılıkla eksiktir: Code::Blocks Linux'ta da vardır, Dev-C++ ise ağırlıklı olarak Windows için kullanılır. Hocanın "hepsi GCC kullanıyor" sözü de büyük olasılıkla Xcode için tam geçerli değildir, Xcode'un varsayılan derleyicisi Clang'dır.

Hocanın ödevlerde en sık gördüğü hata şuydu: C'de `for` başlığının içinde değişken tanımlanmaz. Değişken önce tanımlanır, sonra `for` içinde `i = 0` denir.

> **Ders sonrası not:** Bu kural derste kullanılan C89/C90 için geçerlidir. Büyük olasılıkla C99 ve sonrasında `for (int i = 0; ...)` yazılabilir. Denendi: `gcc -std=c89` bu satırda hata verdi, varsayılan standartla derlendi. Derste C89 istendiği için değişkeni döngüden önce tanımlamak güvenli yoldur.

Linux'ta da Windows'ta da süreç aynıdır: C kodu yazılıp kaydedilir, sonra derleyici çalıştırılır. Slayttaki örnekte `gcc`'ye bir bubble sort kaynak dosyası girdi olarak verilir, çıktı dosyası `bubblesort` adıyla kaydedilir. Bu sırada bağlama da yapılır. Sonra bu dosya çalıştırılır.

```
gcc bubblesort.c -o bubblesort
./bubblesort
```

16. Bir C Programının Temel Yapısı

Her C programında bir `main` fonksiyonu bulunmak ve bu fonksiyonun süslü parantezleri açılmak zorundadır. Programdaki tanımlar ve işlemler bunların içine yazılır. Hocaya göre bir C programının çalışması için olmazsa olmaz olanlar `int`, `main` ve parantezlerdir. `return` ve diğerleri ihtiyaca göre eklenir.

> **Ders sonrası not:** Altyazıda "köşeli parantez" geçiyor. `main` gövdesi için doğrusu süslü parantezdir (`{ }`).

En basit haliyle, içinde `int a = 5;` ve `a = a + 5;` olan bir program da geçerlidir. Ama `printf` eklenecekse `stdio.h` başlık dosyası gerekir.

Hoca ismin "stüdyo" olmadığının altını çizdi: `stdio.h`, standard input output dosyasıdır. Bazılarının "stüdyo" dediğini duymuş olabilirsiniz, ama değildir. `printf` bir çıkış (ekrana yazma), `scanf` bir giriş işlemidir. Hoca bu temel giriş-çıkış işlemlerinin derlenmiş programını getirmek için `include` kullanıldığını söyledi.

> **Ders sonrası not:** `include` büyük olasılıkla derlenmiş kodu değil, `printf` gibi fonksiyonların bildirimlerini getirir. Derlenmiş kod bağlama aşamasında eklenir (14. bölüm).

```c
#include <stdio.h>

int main()
{
    printf("Welcome to my world\n");
    return 0;
}
```

String çift tırnakla yazılır. Hoca bu tuşun kendi klavyesinde `Tab` tuşunun üzerinde olduğunu söyledi.

Soru: Tek tırnaktan iki defa yazsak olur mu?

Cevap: Olmaz. Çift tırnak tuşu kullanılacaktır. Tek tırnağı iki kez yazmak aynı şey değildir.

17. Noktalı Virgül Kuralı

Noktalı virgül ilk zamanlarda herkesin karıştırabileceği bir şeydir, ama birkaç istisna dışında zor değildir. Hocanın kuralı şudur: Bir emir cümlesi kuruyorsanız noktalı virgül kullanırsınız. Bunu anlamak için programı Türkçe okumak yeterlidir.

- "Ekrana *welcome to my world* yaz" bir emirdir, sonuna noktalı virgül gelir.
- "Sıfır döndür" bir emirdir.
- "`int a` değişkenini oluştur" bir emirdir.
- "`a` değişkenine 5 değerini ata" bir emirdir.

Ama "eğer devamsızlık 20'den büyükse" bir emir değildir: Büyükse ne olacak? Yapılacak emirler onun altında gelir ve onlar noktalı virgül içerir. Örneğin "ekrana kaldı yaz" emrinin sonuna noktalı virgül konur. "1'den 10'a kadar" kısmı da emir değildir, emir olan "sayıları ekrana yaz" kısmıdır. Noktalı virgül `main`'in içinde ve diğer fonksiyonların içinde kullanılır, `#include` satırlarında kullanılmaz.

17.1 Yanlış Yerdeki Noktalı Virgül

Noktalı virgül `for` başlığının sonuna konursa döngünün gövdesi orada tamamlanmış olur. Yani boş bir döngü yapılmış olur. Program çalışır, ama ekrana on kez bir şey yazmaz. Hoca bunu Dev-C++'ta canlı olarak denedi:

```c
#include <stdio.h>

int main()
{
    int i = 0;
    for (i = 0; i < 10; i++);
    {
        printf("%d\n", i);
    }
    return 0;
}
```

Derleyici sözdizimi hatası vermez ve program yalnızca `10` yazar. `i` sıfırdan başlar, 9'a kadar gider, sonra 10 olur ve döngü sona erer. Süslü parantezli blok döngüden sonra gelir ve bir kez çalışır. Bu, döngüden çıkınca `i`'nin son değerinin 10 olduğunu da gösterir. Hocanın ifadesiyle noktalı virgül "0'dan 10'a kadar hiçbir şey yapma" demektir.

> **Ders sonrası not:** `gcc -Wall` ile (sürüm 15) bu kod hata vermeden derlendi ve `10` yazdı, ama `-Wmisleading-indentation` uyarısı verdi. Hocanın gösterdiği ortamda böyle bir uyarıdan söz edilmiyor. Bu uyarı büyük olasılıkla tam da bu hatayı yakalamak içindir.

Hoca derleme sırasında ekranın alt kısmında görünenleri de açıkladı. Dev-C++ `gcc` çağırıyor, `deneme.c` dosyasının yolunu veriyor ve `deneme.exe` çıktısını oluşturuyor. MinGW altındaki derleyicinin yolu ve bazı ayarlar da komutta yer alıyor. Dosya kaydedilirken tür olarak C++ değil C seçilmelidir. Bu, çok yapılan bir hatadır.

> **Ders sonrası not:** Hoca `deneme.exe` için "object file" ve `-g3` için "optimizasyon" dedi. `deneme.exe` çalıştırılabilir dosyadır. `-g3` de büyük olasılıkla hata ayıklama bilgisi ekleyen bir seçenektir, optimizasyon değildir.

Soru: O boş bloğun içine kod yazsak çalışır mı?

Cevap: Tabii çalışır. Hoca yukarıda bir `a` tanımlayıp bloğun içine `a = 5;` ve `printf("%d", a);` yazdı. Bu tür bloklara anonim blok da dendiğini, Java'da çok kullanıldığını söyledi.

Soru: Süslü parantezlerin mantığı değişkenlerin geçerlilik alanıyla (scope) mı ilgili?

Cevap: En içte tanımlanan değişken en içte geçerlidir, dışarıda geçerli değildir. Ama hoca bu konunun herkesçe bilinmediğini söyleyip ileriye bıraktı. Buradaki önemli nokta, noktalı virgülün yerini doğru belirlemektir.

> **Hocanın vurgusu:** `for` ve `if` bloklarında, tek satır olsa bile süslü parantez kullanın. "Bir işlem yapacak, gerek yok" demeyin, buna alışın. İleride uzmanlaştıkça kullanmayı bırakabilirsiniz. İlk başlarda hata yapılıyor.

17.2 do-while İstisnası

C'de `if`, `for` ve `while` gibi yapıların başlıklarının sonuna noktalı virgül konmaz. Bunun tek istisnası `do-while`'dır. Normal `while`'da, örneğin `i < 10` olduğu müddetçe çalışacak bir döngüde, yapılacak işler parantezden sonraki bloğa yazılır. Başlığa noktalı virgül konursa kendi kendine dönen boş bir döngü olur.

`do-while`'da ise gövde `do`'dan sonra gelir ve `while` satırı en sona yazılır. Hoca, burada `while` ile `do-while` karışmasın diye sona noktalı virgül konduğunu anlattı:

```
do {
    /* gövde */
} while (i < 10);
```

`for` başlığının içindeki noktalı virgüller ise farklı bir görev görür: Başlangıç, kontrol ve artırma kısımlarını ayırırlar. Hoca `i++`'dan sonra da noktalı virgül konursa çalışıp çalışmayacağını bilmediğini, bakmak gerektiğini söyledi.

> **Ders sonrası not:** Denendi: `for (i = 0; i < 10; i++;)` yazımı `gcc -Wall` ile derleme hatası verdi (`expected ')' before ';' token`). Büyük olasılıkla bütün standart C derleyicilerinde de böyledir.

18. Değişkenler ve RAM Modeli

Hoca değişkenlere hesaplama kuramından girdi. Bir problemi elektronik bir sistemde hesaplarken bellek alanlarına ihtiyaç duyulur. Bu alanlar sıralı ya da rastgele erişilebilir, sonlu sayıda olabilir. Durum sayısına bir kısıt konursa otomata teorisi ortaya çıkar. Programlamada genellikle RAM (Random Access Memory) modeli kullanılır. Çünkü günlük hayattaki birçok problem girdilerden ve bunlara karşılık gelen çıktıların bulunmasından oluşur. Girdileri saklamak için matematikte denklem çözer gibi değişkenler gerekir. İşlemcinin komut yürütebilmesi ve verileri işleyebilmesi için de bunların bellekte saklanması gerekir. Bu yüzden değişkenler için RAM'den alan ayrılır.

Bir değişken tanımlanırken önce tipi, sonra etiketi yazılır. Değiştirilemez olması isteniyorsa tipin önüne `const` eklenir.

Yazım	Hocanın adlandırması
`int a;`	Bildirim (declaration)
`const int b = 0;`	Tanım (definition)
`int c = 0;`	Tanım (definition)
`a = 5;`	Atama (assignment)

Bir şeyin tanımını yapıyorsanız bütün özelliklerini söylemeniz gerekir. `int c = 0;` demek, "`c` diye bir değişken kullanıyorum ve 0 ile başlayacak" demektir. Hem bildirim hem ilk değer

verme (initialization) yapmak tanım anlamına gelir.

Bir öğrencinin "tanımın kendisi atama sayılır mı" sorusuna hoca "bildirim artı atama eşittir tanım diyebiliriz" diye cevap verdi.

> **Ders sonrası not:** C standardının terminolojisi büyük olasılıkla bundan biraz farklıdır: Fonksiyon içindeki `int a;` zaten değişkene yer ayırdığı için bir tanımdır, yalnızca bildirim olan örnek `extern int a;` gibi yazımlardır. İlk değer verme ile atama da büyük olasılıkla ayrı kavramlardır, örneğin `const int b = 0;` geçerlidir ama sonradan `b = 5;` yazılamaz. Derste kullanılan basit şema başlangıç için işe yarar.

18.1 Bir Değişkenin Özellikleri

Hoca `int j = 15;` üzerinden bir değişkenin özelliklerini saydı:

- **Etiket:** Değişkenin adı, burada `j`.
- **Adres:** Değişkenin bellekteki fiziksel adresi.
- **Boyut (size):** Tipine göre değişir.
- **Değer:** Sakladığı değer, burada 15.
- **Pointer:** Adresi tutan değişken. Bunu `scanf`'te kullanıyorsunuz, ayrıntısı pointer konusunda görülecektir.

18.2 Atama Bir Eşitlik Değildir

Programlamaya yeni başlayanlar "`i = j - 2` nasıl olur?" ya da "`i = i + 2` nasıl olur?" diye sorar. Bu bir eşitlik kontrolü değil, atamadır. Önce sağ taraf hesaplanır, sonra sonuç sol taraftaki değişkene aktarılır. Sağ tarafta `i + 2 * 3` gibi bir şey varsa önce çarpma, sonra toplama yapılır. En son atama gerçekleşir.

19. Değişken Adlandırma Kuralları

İsimlendirme kuralları değişkenlerde olduğu gibi sabitlerde, fonksiyonlarda ve `struct` tanımlarında da geçerlidir.

- Türkçe karakter kullanılmaz. Örneğin `ç` ile başlayan `çok` diye bir değişken tanımlanamaz.
- Sembol olarak yalnızca alt çizgi (_) kullanılır. Soru işareti ve ünlem işareti olmaz. Bir ad alt çizgi içerebilir, alt çizgiyle de başlayabilir.
- Rakam içerebilir, ama rakamla başlayamaz.
- C büyük-küçük harfe duyarlıdır (case sensitive). `sum`, `Sum` ve `SUM` üç ayrı değişkendir. Pascal'da böyle değildi. Hoca bunu isimlendirmede kolaylık sağladığı için güzel buluyor.
- Ayrılmış anahtar kelimeler (reserved keywords) kullanılamaz: `int`, `for`, `auto`, `else`, `long`, `typedef` gibi. Bunları ezberlemek gerekmez, IDE bunları farklı renkte gösterir. Normal şartlarda kimse durduk yere `typedef` diye değişken tanımlamaz.
- Ad, üzerinde çalışılan veriyle anlamlı bir ilişki taşımalıdır.

  > **Ders sonrası not:** Hoca `printf` adının da kullanılamayacağını söyledi. `printf` büyük olasılıkla anahtar kelime değil, kütüphanedeki bir addır. Denendi: `int printf = 3;` satırı `gcc -Wall` ile hata vermedi. Ama aynı kapsamda artık `printf` fonksiyonu çağrılamayacağı için bu addan kaçınmak doğrudur.

20. Veri Tipleri

Değişken isimlerinden sonra hoca veri tiplerine geçti.

- **`void`:** Hoca bunu "global tip, birbirine dönüştürülebilir bir tip" diye nitelendirdi. Ayrıntısı muhtemelen sonraki dönemde görülecektir.
- **Skaler tipler:** Tek bir bileşeni, tek bir büyüklüğü ifade eder. Yalnızca tam sayı, yalnızca kayan noktalı sayı ya da yalnızca karakter gibi. Pointer'lar ve enumeration da bu gruptadır.
- **Birleşik (aggregate) tipler:** Skaler tiplerin bir araya getirilmesiyle oluşur.

Aritmetik tipler içinde tam sayılar (`int`, `short`, `long`) ve kayan noktalı sayılar (`float`, `double`) vardır. Hoca bunları ezberlemenin önemli olmadığını, örnek yaptıkça karşımıza geleceğini söyledi.

Pointer tipleri için hoca şunu belirtti: `a`'nın adresi farklı bir veri tipidir ve yıldızla tanımlanır. `int`, `int *` ve `int **` birbirinden farklı tiplerdir, birbirine dönüştürülemez.

> **Ders sonrası not:** Bu tipler gerçekten farklıdır, ama "dönüştürülemez" ifadesi büyük olasılıkla "kendiliğinden, güvenle dönüştürülmez" anlamında alınmalıdır. C'de açık dönüşümle (cast) pointer tipleri arasında dönüşüm yazılabilir.

20.1 Karmaşık Sayı Örneği

C'de karmaşık sayı diye bir tip olmadığını söyleyen hoca, bunun programcı tarafından tanımlanabileceğini anlattı. Karmaşık sayının reel ve sanal iki bileşeni vardır. Önce "bir `int`" diye başladı, sonra reel sayı için `float` olması gerektiğini söyledi. İki `float` birleştirilip yeni bir tip oluşturulur:

```c
#include <stdio.h>

struct kompleks {
    float re;
    float im;
};

int main()
{
    struct kompleks c;
    c.re = 1.0f;
    c.im = 2.0f;
    printf("%f %f\n", c.re, c.im);
    return 0;
}
```

Aynı iş iki ayrı `float` değişkeniyle de yapılabilir: Birini reel, öbürünü sanal kabul edersiniz, ikisi ayrı ayrı yürür. Ama problem çözerken bu çok kullanışlı olmaz. Yeni tip, veriler arasındaki bütünlüğü sağlar. `c` değişkeni oluşturulur ve bileşenlerine nokta operatörüyle erişilir. Bu konu `struct` konusunda görülecektir.

> **Ders sonrası not:** Büyük olasılıkla C99'dan itibaren dilde `_Complex` tipi de vardır. Derste kullanılan C89 açısından hocanın söylediği geçerlidir. Yukarıdaki kod derste sözlü anlatılan yapıdan kuruldu, atanan değerler yalnızca kodun çalıştığını göstermek için konuldu.

20.2 Temel Tipler ve Niteleyiciler

Temel veri tipleri karakterler, `int`, `float`, `double` ve enumeration'dır. Başka dilleri bilenler string'in olmadığını fark edebilir: C'de `string` tipi yoktur, karakterler dizi olarak kullanılır. Temel tiplerden `char c[...]` ya da `int a[...]` gibi diziler oluşturulur.

Tiplerin önüne niteleyiciler (qualifier) gelebilir. `short` ya da `long` ile boyut küçültülüp büyütülebilir, `signed` ya da `unsigned` ile sayıların işaretli olup olmadığı belirtilir. Hocanın yorumu şuydu: C'de her şeyi kendiniz yaparsınız. C ayrı bir `uint` tipi tanımlamamış, "ben `int`'i bilirim, istersen önüne `signed` ya da `unsigned` getir" demiştir.

> **Ders sonrası not:** Hoca `uint` tipi için Java ve C#'ı örnek gösterdi. `uint` büyük olasılıkla yalnızca C#'ta vardır, Java'da bu adla işaretsiz bir tam sayı tipi yoktur.

20.3 Değer Aralıkları

`char` 1 byte, yani 8 bittir. Bununla $2^8 = 256$ farklı sembol kodlanabilir.

- `unsigned char`: 0 ile 255 arası.

- `signed char`: Bir bit işarete gider. $-2^7 = -128$ ile $2^7 - 1 = 127$ arası.

`short` genellikle 2 byte, yani 16 bittir.

- `short`: $-2^{15} = -32768$ ile $2^{15} - 1 = 32767$ arası.
- `unsigned short`: 0 ile $2^{16} - 1 = 65535$ arası.

Hoca bu kısmı anlatırken önce "18 bit" ve "2^{17}" dedi, sonra hesabı $8+8 = 16$ bit olarak düzeltti. Slaytta `unsigned short` üst sınırının yanlış yazıldığını da fark edip 65535 olması gerektiğini söyledi.

`int` 4 byte, yani 32 bittir. `unsigned int` için üst sınır $2^{32} - 1 = 4294967295$'tir. `long` için hoca 8 byte dedi ve $2^{64} - 1$ üst sınırını gösterdi. `float` ve `double` kayan noktalı sayılardır, IEEE notasyonundaki tek ve çift duyarlıkla ilişkilidir. Hoca C'deki tam karşılıklarını bilmediğini söyledi. Biri 4, öbürü 8 byte'tır ve 8 byte olan daha ayrıntılı değer tutar.

Bu boyutlar işletim sistemine ve derleyiciye göre değişebilir. "Bende `int` 8 byte görünüyor" diyen biri de yanlış sayılmaz. Burada genel durum konuşulmaktadır.

21. Tiplerin Boyutları ve sizeof

Tiplerin bilgisayardaki boyutunu görmek için `sizeof` kullanılır. Hoca fonksiyon gibi göründüğünü, ama bir operatör olduğunu söyledi. Slayttaki programı Dev-C++'a kopyalayıp çalıştırdı. Aşağıdaki kod aynı fikri gösterir:

```c
#include <stdio.h>

int main()
{
    printf("char: %d\n", (int) sizeof(char));
    printf("short: %d\n", (int) sizeof(short));
    printf("int: %d\n", (int) sizeof(int));
    printf("long: %d\n", (int) sizeof(long));
    printf("long long: %d\n", (int) sizeof(long long int));
    printf("unsigned char: %d\n", (int) sizeof(unsigned char));
    printf("float: %d\n", (int) sizeof(float));
    printf("double: %d\n", (int) sizeof(double));
    printf("long double: %d\n", (int) sizeof(long double));
    return 0;
}
```

> **Ders sonrası not:** Slayttaki kodda `sizeof` sonucu doğrudan `%d` ile yazdırılıyordu. `gcc -Wall` ile (64 bit Linux) bu yazım uyarı verdi, çünkü sonuç `long unsigned int` tipindeydi. Uyarısız derlensin diye yukarıda `(int)` dönüşümü eklendi. Bu dönüşüm 27. bölümde anlatılan açık dönüşümdür.

Hocanın ekranında `long double` 16 byte, `long int` 4 byte ve `long long int` 8 byte çıktı. Canlı denemeler şöyle ilerledi:

- `long long double` denendi ve derleyici kabul etmedi.
- `long long int` denendi ve 8 byte verdi. Hoca `int` 4 byte iken bunun 16 byte olmadığına dikkat çekti.
- Bir öğrencinin önerisiyle `long long long` denendi ve derleyici "too long for GCC" hatası verdi.
- Tek `long` ile `int`'in aynı 4 byte olduğunu gören hoca, derleyicinin bunu muhtemelen algılamadığını söyledi.

> **Ders sonrası not:** Derleyici `long` nitelemesini algılamıyor değildir. Windows'ta (64 bit dahil) `long` büyük olasılıkla 4 byte'tır. 64 bit Linux'ta ise 8 byte'tır. Burada `gcc -Wall` ile yapılan denemede `long` 8, `long long` 8, `long double` 16 byte çıktı. `long long double` için `both 'long long' and 'double'`, `long long long` için `'long long long' is too long for GCC` hatası alındı.

21.1 Operatör ile Fonksiyonun Farkı

Soru: Operatör ile fonksiyonun farkı nedir?

Cevap: `printf` bir fonksiyondur. Fonksiyonda, bulunduğu yere gidilir, iş yapılır ve geri dönülür. Örneğin dördüncü satırdaysak fonksiyona gidilir, fonksiyon biter ve beşinci satıra gelinir. Operatörde ise `+`, `-`, `*` ya da `sizeof` için hiçbir yere gidilmez. Operatörün tanımlı olması gerekir ve iş bulunulan yerde yapılır. `c = a + b` yazıldığında derleyici `+`'nın solundaki ve sağındaki toplanabilir verileri toplar. Sonucu da eşittir operatörü sol taraftaki değişkene aktarır. `sizeof` tek operand alan bir operatördür.

22. Bildirim, Tanım ve Sabitler

Hoca slaytta yalnızca tip ve isim belirten yazımların bildirim olduğunu tekrarladı. Değer ataması da yapılıyorsa bu ilk değer vermedir ve bildirimle birlikte tanım sayılır. Hocaya göre derleme aşamasında bildirim yalnızca değişkenin kullanılacağını söyler. İlk değer verilmemiş değişkenler, ilk değer verilmiş değişkenler ve sabitler bellekte farklı yerlerde saklanır. Bu, İşletim Sistemleri dersinde anlatılacaktır.

> **Ders sonrası not:** Bu ayrım büyük olasılıkla global ve `static` değişkenler ile sabitler için geçerlidir. Fonksiyon içindeki yerel değişkenler ise ilk değer verilsin verilmesin genellikle yığında (stack) tutulur.

Bir değerin değiştirilemeyeceği biliniyorsa önüne `const` yazılır. C'de sabit tanımlamanın ikinci yolu `#define`'dır:

```c
#include <stdio.h>

#define PI 3.14

int main()
{
    const float pi = 3.14f;
    float r = 2.0f;
    printf("%f %f\n", PI * r * r, pi * r * r);
    return 0;
}
```

İkisi arasındaki farkı hoca bir alan hesabı üzerinden anlattı. `const` bir değişkenin değeri çalışma sırasında bellekten getirilir. `#define` ise bir makrodur: Program derlenirken `PI` yazılan her yere değer birebir kopyalanır, yani bellekten getirme gibi bir durum olmaz. İkisi de değiştirilemez, ikisi de sabit olarak kullanılır.

Yazım farkları da vardır. `#define` satırında tip yazılmaz, bu yüzden `PI`'nin karşısına bir string ya da karakter yazılırsa çarpma ve bölme yapılamaz. Noktalı virgül kullanılmaz. Büyük harfle yazılması ise bir gelenektir, makro olduğu belli olsun diye. Normal değişkenler genellikle küçük harfle yazılır. Makrolar daha ayrıntılı olarak ileride ele alınacaktır.

> **Ek bilgi (derste anlatılmadı):** `#define` satırlarını işleyen birim büyük olasılıkla derleyicinin kendisi değil, derlemeden önce çalışan önişlemcidir (preprocessor). `#include` da aynı aşamada işlenir. Önişlemci metin üzerinde çalıştığı için makroda tip denetimi yapmaz. `#define` satırının sonuna noktalı virgül konmamasının nedeni de büyük olasılıkla budur: Noktalı virgül de kopyalanan metnin parçası olurdu.

23. Literaller

Literal, temel bir değerin doğrudan yazılmasıdır. Hoca bunu Assembly dersinde görülecek immediate adresleme moduna benzetti: Bir değişkeni başka bir değişkene değil, temel bir büyüklüğü doğrudan atamak. `22` sabit bir sayıdır ve doğrudan bir değişkene atanabilir.

Tipi belirtmek için sona harfler eklenir: `u` işaretsiz, `l` uzun, `f` ise `float` demektir. Ondalıklı bir sayı yazıldığında normalde `double`'dır. `float` olarak atanması isteniyorsa sonuna `f` eklenir. Bir de `e3L` biçiminde yazılan `long double` gösterimi vardır. Hoca slayttaki kodların birbirinden bağımsız parçalar olduğunu, tip dönüşümünün burada konuşulmayacağını söyledi.

Tam sayı sabitleri onluk, sekizlik ya da on altılık yazılabilir. On altılık yazımda başa `0x` konur: `0x4` ya da `0x21`. `0x21` değeri onluk sistemde 33'tür, dönüşüm arka planda yapılır.

Kayan noktalı sayılarda virgül değil nokta kullanılır. Bu dönem Sayısal Analiz'de virgüle alışanlar dikkat etmelidir, yoksa hata alınır. `4,0` yazılırsa iki ayrı değer gibi olur.

Soru: Sekizlik sayı `printf` ile nasıl yazılır?

Cevap: Hoca sekizliğin nasıl yazdırıldığını bilmediğini, belki bir belirteç olduğunu, araştırıp paylaşacağını söyledi. On altılık yazıldığında derleyici onu doğrudan on altılık olarak algılar.

> **Ders sonrası not:** Sekizlik yazdırma için büyük olasılıkla `%o` belirteci kullanılır, sekizlik literal de başına `0` konarak yazılır. Denendi: `printf("%d %o", 015, 015);` ekrana `13 15` yazdı. Yani hocanın andığı `015` ifadesi büyük olasılıkla onluk 13'tür.

23.1 Geçerli ve Geçersiz Yazımlar

Hoca slayttaki listede virgül içeren yazımların geçersiz olduğunu, üstel kısmın da yalnızca belirli biçimde yazılabileceğini anlattı. Üstel gösterimde `e`'den sonraki sayı 10'un kuvvetidir: `3e2` için $3 \cdot 10^{2}$, `3e-2` için $3 \cdot 10^{-2}$.

Soru: Listedeki 8 neden öyle işaretlenmiş?

Cevap: Hoca kayan noktalı yazım için `8.0` denmesi gerektiğini, slaytın bu satırında muhtemelen bir hata bulunduğunu söyledi.

> **Ders sonrası not:** `8` büyük olasılıkla kayan noktalı değil tam sayı sabitidir. Ama `double x = 8;` yazımı geçerlidir, çünkü değer atamada gizlice dönüştürülür. Yani yazım hatası değil, tip farkı söz konusudur.

Soru: `3e2`'deki `e`, on altılık sistemle mi ilgili?

Cevap: Hayır, 10 üzeri 2 gibidir. Hoca bunu `printf("%f", 3e2)` ile denedi.

```c
#include <stdio.h>

int main()
{
    printf("%f %f\n", 3e2, 3e-2);
    return 0;
}
```

Çıktı `300.000000 0.030000` olur. Hoca noktadan sonraki sıfırlar yüzünden bunun 300 milyon sanılmaması gerektiğini söyledi: Bu 300 tam, ardından ondalık kısımdır.

Hoca programlamada Türkçe alışkanlıkları bırakmak gerektiğini de ekledi. Java'da Türkçe karakterler değişken adı olarak kullanılabilse de ortak dil olarak İngilizce kullanılmasını tercih ediyor.

24. Giriş ve Çıkış Fonksiyonları

Giriş ve çıkış fonksiyonları kullanıcıyla etkileşim içindir. Veri işlenecekse önce alınmalıdır, sonuç da kullanıcıya yansıtılmalıdır. `printf` çıkış, `scanf` giriş içindir. Bunların yanında bir string'i ekrana basan `puts`, klavyeden string okuyan `gets` ve tek karakter okuyan `getchar` vardır.

24.1 Format Belirteçleri

C birçok şeyi kendisi yapmaz, ekrana ne yazılacağını ya da kullanıcıdan ne alınacağını programcı söyler. Bunun için format belirteçleri (format specifier) kullanılır.

Belirteç	Tip
`%c`	karakter
`%d`	tam sayı
`%s`	string (karakter dizisi)

Belirteç	Tip
`%ld`	`long`
`%u`	`unsigned`
`%lu`	`unsigned long`
`%f`	`float`
`%lf`	`double` (long float)
`%x`	on altılık

Hoca `%ld` için "long decimal" diye düşünmeyi önerdi. `double` için `%d` kullanılmaz, çünkü `%d` tam sayıya aittir. Belirteçler metnin herhangi bir yerine yazılabilir. Her belirteç sırayla virgülden sonraki parametrelerle eşleşir:

```
printf("metin %c metin %f metin\n", 'a', 3.5);
```

Bu arada karakterler tek tırnakla, string'ler çift tırnakla yazılır. Hoca başka dillerde `0, 1, 2, 3` gibi etiketlerle parametreye işaret edilebildiğini, C'de ise bu işin sıraya bırakıldığını söyledi. Hoca on altılık gösterimi de canlı olarak denedi.

24.2 `%u` ile Eksi Sayı

Hoca `printf("%u", -5);` satırını denedi. Yazılacak değer ile belirteç arasında mantıksal bir uyumsuzluk vardır. Ekrana 4294967291 yazıldı. Bu değer, slayttaki tablodaki `unsigned int` üst sınırına göre hesaplanır:

$$2^{32} - 5 = 4294967291$$

> **Ders sonrası not:** Hoca hesabı üst sınırdan 5 çıkarmak gibi anlattı, ama $4294967295 - 5 = 4294967290$ olur. Doğrusu $2^{32} - 5$'tir. Ekrandaki 4294967291 değeri de bununla uyuşur. Python ve `gcc` ile doğrulandı.

Soru: `unsigned int` tanımlayıp `%d` ile yazdırırsak ne olur?

Cevap: Hocaya göre `%d` ile yazılan değer çıktıda `%d`'nin türüne dönüşür. Küçük bir tip büyük bir tipe dönüşür, büyük tip küçüğe dönüşmez. `unsigned int b = 10;` için `%d` yine 10 yazar.

Soru: Bellekte saklanırken yine `unsigned int` boyutunda mı?

Cevap: 4 byte ise yine 4 byte saklanır. Yazılan değer bellekte değişmez. `int b = -10;` için `b` bellekte -10 olarak kalır. `%u` ile yazınca ekranda dönüştürülmüş değer görülür, `%d` ile yazınca yine -10 görülür. Hoca bunu aynı satırda ikisini yazdırarak gösterdi:

```
#include <stdio.h>

int main()
{
    int b = -10;
    printf("%u %d\n", b, b);
    return 0;
}
```

Çıktı `4294967286 -10` olur ($2^{32} - 10 = 4294967286$). Denemeler sırasında derleyici değişkenin iki kez tanımlandığı ya da hiç tanımlanmadığı hatalarını verdi. Hoca hata mesajlarından korkmamak gerektiğini söyledi.

> **Hocanın vurgusu:** Çok çok hata yapın, ancak öyle öğrenirsiniz.

> **Ders sonrası not:** `int` ile `unsigned int` büyük olasılıkla aynı boyuttadır, biri öbüründen "küçük" sayılmaz, fark bitlerin nasıl yorumlandığıdır. `printf` da büyük olasılıkla değeri dönüştürmez, belirtece göre aynı bitleri farklı okur. Bu yüzden iki yönde de değer farklı görülebilir.

24.3 printf Nasıl Çalışır?

Hoca `printf`'in ekrana bastıracağı şeyi string olarak bastırdığını anlattı. Prototipine bakılırsa bir karakter dizisi tanımlı olduğu görülür. Format belirteçleriyle bir bağlama yapılır: `a eşittir %d` gibi bir ifadede `%d`'nin yerine ilk parametre yazılır. Oluşan string ekrana gönderilir. `scanf`'te ise klavyeden girilen şey string biçiminde alınır, `%d` ise tam sayıya dönüştürülür, tabii dönüştürülebiliyorsa. `%c` ve `%d` yazmanın nedeni budur: Hangi tipe dönüştürüleceğini belirtmek.

Hoca bir de toplam örneği verdi. `printf` ile "the sum of a and b is equal to %d" yazılıp parametre olarak `a + b` veriliyordu. Önce `a + b` hesaplanır, sonuç metne eklenir. Ortalama örneğinde de "the average of this array is equal to %d" gibi bir metinde `avg` değişkeninin değeri yerine konur.

> **Ders sonrası not:** Hoca burada "string'in yanına bir şey eklenince her şey string'e dönüşür" dedi. Operatör önceliği olmasaydı `a` ve `b` yan yana yazılıp 35 gibi bir şey görüleceğini söyledi. Bu açıklama büyük olasılıkla Java'daki `"..." + a + b` davranışına aittir. C'de string ile sayı `+` ile birleştirilmez. `a + b` yalnızca `printf`'e verilen bir argümandır, çağrıdan önce hesaplanır ve `%d`'nin yerine yazılır.

24.4 scanf ve Adres İşareti

`scanf` için hoca "belleğe yazdırma işlemi" dedi: Kullanıcıdan alınan veri bellekteki bir adrese yazılır. Bu yüzden string, yani karakter dizisi hariç bütün değişkenlerin önüne `&` konur. Hoca şimdilik bunu ezberlemenin yeterli olduğunu, nedenini pointer konusunda anlatacağını söyledi. Karakter dizilerinde `&` kullanmaya gerek olmadığını, kullanılsa da çalışacağını belirtti.

> **Ders sonrası not:** Hoca `&` konmazsa hata alınacağını söyledi. Denendi: `scanf("%d", a);` `gcc -Wall` ile derleme hatası değil uyarı verdi. Yani hata büyük olasılıkla çalışma sırasında ortaya çıkar. Karakter dizisinde `&k` yazımı da uyarı verdi (`char (*)[9]`). Doğru yazım büyük olasılıkla `scanf("%s", kelime);` biçimidir.

24.5 puts, gets ve Boşluk Sorunu

Bir kelime `&` kullanmadan `gets(kelime);` ile okunabilir, `puts(kelime);` ile yazılabilir. Hoca slayttaki `puts`'un `printf` ile aynı işi gördüğünü söyledi.

> **Ders sonrası not:** `puts` büyük olasılıkla ek parametre almaz ve sona alt satır ekler, ek parametre `printf`'e özgüdür. `gets` de dizi boyutunu denetlemediği için büyük olasılıkla güvensizdir ve C11'den çıkarılmıştır. Denendi: `gets` kullanan kod `gcc -Wall` ile (sürüm 15, varsayılan standart) hata verdi. Yerine büyük olasılıkla `fgets(kelime, sizeof(kelime), stdin)` kullanılmalıdır.

Soru: `scanf` boşluk görene kadar alıyor. Ad ve soyadımı yazınca yalnızca ilk adımı alıyor.

Cevap: `%s` varsayılan olarak boşluk karakterlerini (white space) kapsamaz. Bunun için format içine bir ifade yazılır:

```c
#include <stdio.h>

int main()
{
    char kelime[50];
    scanf("%[^\n]", kelime);
    printf("%s\n", kelime);
    return 0;
}
```

Buradaki `^` işareti "değil" demektir, `\n` de Enter'a karşılık gelir. Yani "alt satır dışındaki her şeyi al", başka bir deyişle Enter'a kadar yazılan her şeyi al. Hoca `^A` yazılırsa `A` görülene kadar alınacağını da söyledi. "Ad Soyad" girdisiyle yapılan denemede `%s` yalnızca `Ad`, `%[^\n]` ise `Ad Soyad` yazdırdı.

Ders sonrası not: Hoca buna düzenli ifade (regular expression) dedi. C'de bunun adı büyük olasılıkla "scanset"tir ve yalnızca köşeli parantez içindeki karakter kümesini tanır, tam bir düzenli ifade değildir.

25. Gizli Tip Dönüşümü

Birbirine dönüştürülebilir iki tip üzerinde işlem yapılırken iki tür dönüşüm vardır. Bunlardan biri gizli (implicit) dönüşümdür ve aritmetik işlemlerde, tip genişletme (integral widening conversion) durumunda karşımıza çıkar.

İlk örnekte 2 tam sayı, 1.2 ise `float` ya da `double`'dır. Sonuç otomatik olarak `double`'a döner. Sonuç 2.4'tür. Atama yapılan değişken `double` ise 2.4 saklanır. Değişken `int` ise ondalık kısım gider ve 2 saklanır.

Ders sonrası not: Hoca burada "yuvarlama" dedi. Ondalıklı bir değer `int`'e atanınca büyük olasılıkla yuvarlanmaz, ondalık kısmı atılır (sıfıra doğru kesme).

Sonraki örnekler iki tam sayının bölümüyle ilgilidir:

- `2.0 + 1 / 2`: `1 / 2` iki tam sayının bölümüdür ve 0 verir. Sonuç $2.0 + 0 = 2.0$ olur.
- `2.0 + 1.0 / 2`: `1.0 / 2 = 0.5` verir. Sonuç $2.0 + 0.5 = 2.5$ olur.
- `-3 / 4 + 2.5`: `-3 / 4` tam sayı bölmesi olarak 0 verir. Sonuç $0 + 2.5 = 2.5$ olur.

-0.75 elde etmek istiyorsak işlemde bir `double` parça bulunmalıdır: `3` yerine `3.0` yazılır ya da `1.0` ile çarpılır. Bu durumda `-3.0 / 4 + 2.5` için sonuç $-0.75 + 2.5 = 1.75$ olur. Bu değerler `gcc` ile denendi ve doğrulandı.

Derleyici bir `+` işaretinin solundaki ve sağındaki tiplere bakar. `2 + 3.5` örneğinde sağdaki `double`, soldaki `int` olduğu için ikisi de `double` gibi düşünülür: 2.0 ve 3.5. Atama yapılacaksa belirleyici olan değişkenin tipidir. `A = 2 + 3.5` yazılıp `A` `int` tanımlanırsa 5,5 görülmez, doğrudan 5 görülür.

Ders sonrası not: Hoca bu toplamın sonucunu önce "3.5" olarak söyledi. Doğrusu $2.0 + 3.5 = 5.5$'tir. Hemen sonra kendisi de 5,5 üzerinden devam ediyor.

26. Operatörler ve Operandlar

İşaretlere operatör, kullandıkları verilere operand denir. Operatörler operand sayısına göre üçe ayrılır:

- **Tekli (unary):** Tek operand alır. `sizeof`, `++`, `--` gibi.
- **İkili (binary):** İki operand alır, biri sola biri sağa gelir. `+`, `-`, `==`, `!=`, `>=`, `<=` gibi. Hoca en kalabalık grubun bu olduğunu söyledi.
- **Üçlü (ternary):** Soru işareti operatörüdür. Bir şart yazılır, şart doğruysa ilk işlem, yanlışsa ikinci işlem yapılır: `şart ? a : b`. Hoca üç operand aldığını gösterdi. Operandların tiplerinin uyumlu olması gerekir.

Hoca başka tekli operatör aklına gelmediğini söyledi, `free`'yi anıp onun fonksiyon olduğunu ekledi.

Ders sonrası not: Altyazıda hoca "iki tane operatör olacak" dedi, kastedilen iki operanddır.

27. Büyük Tipten Küçük Tipe Dönüşüm ve Açık Dönüşüm

Küçük bir tipten büyük bir tipe dönüşümde sıkıntı yoktur. Hoca bunu şöyle gösterdi: `int`'teki pozitif bir değer `long int`'e de sığar, `long double` olarak da rahatça saklanır.

Büyük tipten küçük tipe dönüşüm C'de yasak değildir, ama kayıp vardır. Hocaya göre Java'da bu tür bir atama yapılamaz: Derleyici derleme zamanında hata verir, program derlenmez bile. C'de ise program gayet güzel derlenir ve "bir sonuç alıyorum ama başka değerler geliyor" durumu ortaya çıkar. Çünkü C burada yetkiyi programcıya bırakır.

> **Ders sonrası not:** Hoca Java için `long double`'dan `double`'a atamayı da örnek verdi. Java'da `long double` tipi büyük olasılıkla yoktur, ama `double`'dan `int`'e atama için söyledikleri geçerlidir.

Hocanın örneği `unsigned int` üst sınırı 4294967295'in 3 eksiğiydi: 4294967292. Bu değer `int`'e dönüştürülürse `int`'te tanımlı olmadığı için, çünkü bir bit işarete gittiği için, negatif ya da farklı bir sayı görülür. Hoca hangi sayı olduğunun şimdilik önemli olmadığını söyledi.

> **Ders sonrası not:** `gcc -Wall` ile yapılan denemede bu değer `int`'e atanınca -4 görüldü ($4294967292 - 2^{32} = -4$). Bu sonuç büyük olasılıkla derleyiciye bağlıdır.

Bunlar görünmeyen dönüşümlerdir. Bir de açık (explicit) dönüşüm vardır. Kod yazarken "önce `j`'yi `float`'a dönüştür, sonra `k`'ya böl" denir:

```c
#include <stdio.h>

int main()
{
    int j = 3, k = 2;
    float f;
    f = j / k;
    printf("%f\n", f);
    f = (float) j / k;
    printf("%f\n", f);
    return 0;
}
```

Çıktı önce `1.000000`, sonra `1.500000` olur. Birinci satırda 3 bölü 2 tam sayı bölmesi olarak 1 verir. İkincisinde `j` `float`'a dönüştürüldüğü için 1.5 elde edilir ve `f` `float` olduğundan rahatça saklanır.

28. Değişkenler Bellekte Bitişik Tutulsa Daha mı Hızlı Olur?

Dersin sonunda bir öğrenci şunu sordu.

Soru: Dizi elemanlarını RAM'e sırayla yerleştiriyoruz, ama farklı değişkenler rastgele yerlere yerleşebiliyor. `A` ve `B` değişkenlerini de bitişik yerleştirsek daha hızlı erişmez miyiz?

Cevap: Hoca hizalama (alignment) diye bir şeyin `struct`'larda var olduğunu, normal değişkenlerde olup olmadığını bilmediğini söyledi. Hızla ilgili olarak komut yürütme aşamasını anlattı. Adres çözme (address decoding) yapılır: Getirilecek operandın adresi adres yolu üzerinden belirtilir, veri ise veri yolu üzerinden getirilir.

Hoca `A = A + B` örneğini verdi. Bu, `ADD AX, BX` gibi bir komuttur. Komut önce işlemciye gelir ve toplama olduğu anlaşılır. `A`'nın değerinin `AX`'e getirilmesi için bir saat aralığı üretilir. Bir saat sinyali daha gelince ikinci operand `B` getirilir. Sonra işlem tamamlanır. Dolayısıyla "gitmişken onu da getireyim" diye bir durum yoktur.

Komutun ve verilerin tek seferde getirilmediğini adresleme modlarıyla açıkladı. Moda göre belleğe bir, iki, üç kez gidip gelinir ve her gidiş geliş saat çevrimleriyle olur.

- `ADD AX, 5`: `AX`'in değeri kullanılır, 5 komutla zaten gelmiştir.
- `ADD AX, BX`: İki yazmaç kullanılır.
- Belleği gösteren biçimde: `BX`'teki değer bir adres gösterir, önce komut, sonra bellekteki adresin içeriği gelir, burada belleğe iki üç kez gidip gelinir.

Bu yüzden hocaya göre `A` ile `B`'nin yerleri bildiği kadarıyla çok önemli değildir. Dizilerde ise durum başkadır. Dizi doğrudan makine dilinde tanımlı değildir, indeks yazmaçlarıyla gerçekleştirilir. Program yazarken bir elemanın sonraki elemanla ilişkisine göre algoritma tasarlanır, bu yüzden elemanların bitişik olması önemlidir. Normal değişkenlerin bitişik olmasına gerek yoktur. Hoca paralel işlemler gibi konularda ayrıca bakmak gerektiğini ekledi.

> **Ek bilgi (derste anlatılmadı):** Günümüz işlemcilerinde büyük olasılıkla önbellek (cache) devreye girer. Bellekten bir değer okunurken çevresindeki baytlar da blok halinde önbelleğe alınır. Bu yüzden bellekte yakın duran verilere erişim uygulamada

daha hızlı olabilir. Yani öğrencinin sorusundaki sezgi önbellekli sistemlerde büyük olasılıkla tamamen boşa çıkmaz, ama fark bir tek saat çevrimi meselesi değildir.

Dersin Toparlaması

Bu derste önce akış diyagramı sembolleri, örnekler üzerinden adım adım değişken izleyerek tekrar edildi. `if`'in kollarının aşağıda birleştiği, döngülerin şart doğru oldukça döndüğü, `do-while`'ın en az bir kez çalıştığı görüldü. Ardından algoritmanın hayat bulması için gereken programlama dillerine geçildi: makine dili ve assembly, yüksek seviyeli diller, derleyici ve yorumlayıcı, taşınabilirlik. C'nin neden güçlü ama tehlikeli olduğu, yazılım geliştirme sürecinin aşamaları ve modülerlik anlatıldı. Kaynak kodun obje dosyasına, oradan bağlayıcı ile çalıştırılabilir dosyaya ve prosese giden yol izlendi. C tarafında noktalı virgül kuralı, bildirim ve tanım, adlandırma kuralları, veri tipleri ve boyutları, `sizeof`, literaller, format belirteçleri, `printf` ve `scanf` ile gizli ve açık tip dönüşümleri işlendi.

Hoca bir sonraki derste bir örnek program üzerinden derleme sürecini ve obje dosyasının içini göstereceğini söyledi. Hoca artık dersin asıl konularına da gireceğini belirtti: Yaklaşık değer bulma, örneğin π ve e^x hesabı, Gauss toplamı, ardışık sayıların çarpımı ve Fibonacci sayıları. Boş zamanda, Sayısal Analiz dersindeki konuların algoritmalarını yazmayı önerdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.